Multi-Machine Navigation

1. Course Overview

- Run the example program and use two MicroRosV2 robots to navigate in the same map.
- This tutorial demonstrates two MicroRosV2 robots; robot1: PTZ servo version (ptz), robot2: fixed camera version (no_ptz);
- There is no limit on the number or model type of robots actually used. Fill in the `robot_names` and `camera_types` startup parameters according to the actual situation.

2. Preparation

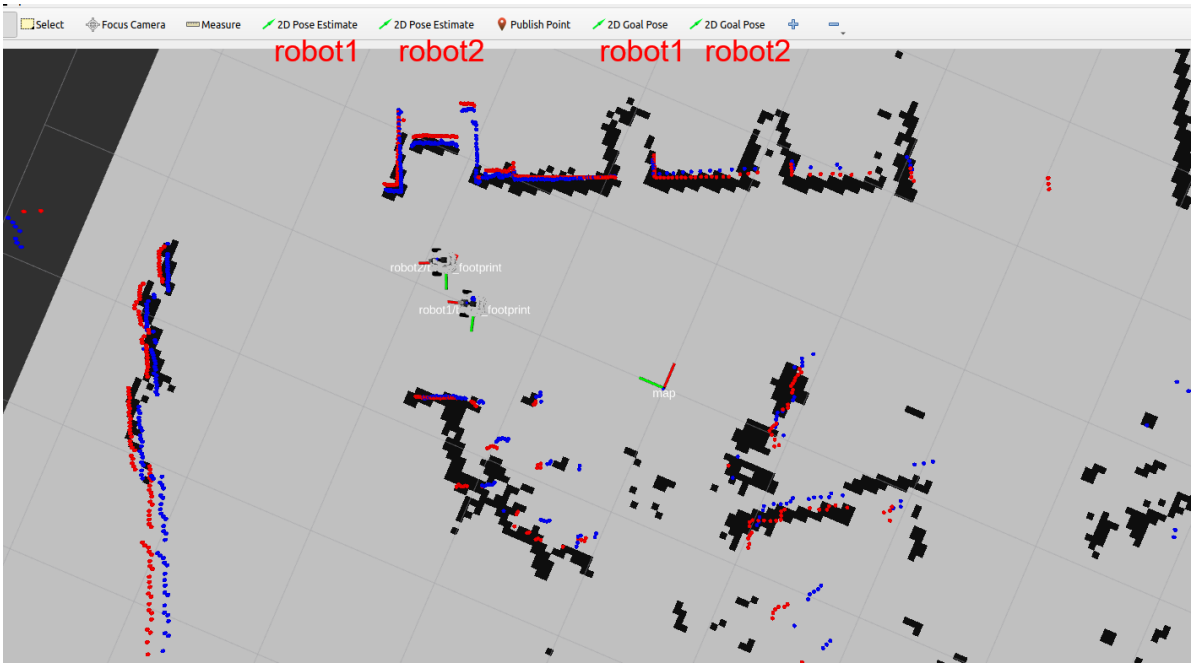
- Build at least one grid map. Refer to the content of the previous chapter, LiDAR Gameplay.
- Complete the prerequisite courses Multi-machine Handle Control and Multi-machine Keyboard Control, and complete the Agent proxy configuration and Domain ID configuration for the robot chassis.

3. Run the Case

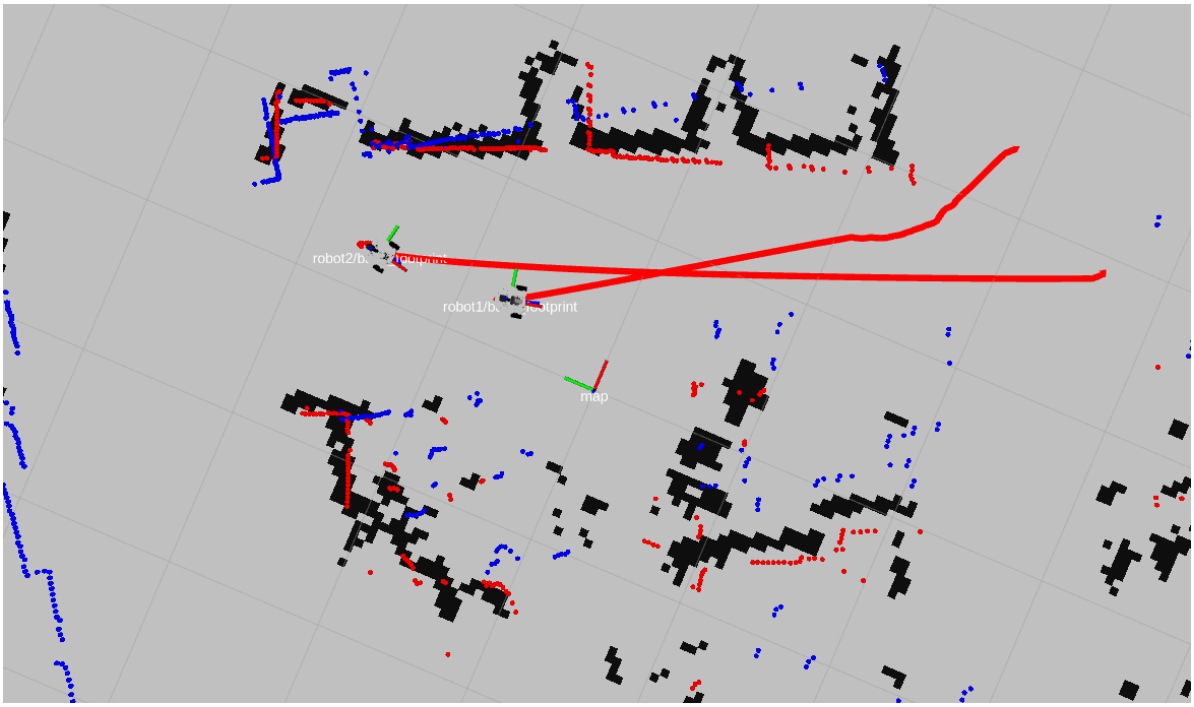
```
ros2 launch microros_v2_bringup multi_robot_demo.launch.py demo:=navigation
robot_names:=robot1,robot2 camera_types:=ptz,no_ptz
```

> [!IMPORTANT]
>
> Parameter description
>
> - robot_names: List of robot names, separated by English commas. For example, robot1,robot2 means two MicroRosV2 robots.
> - camera_types: List of camera model types, separated by English commas. For example, ptz,no_ptz means the camera types of the two robots; the order matches the order of robot_names.

- Use **2D Pose Estimate** to initialize the pose of robot1 and robot2 respectively. There are two "2D Pose Estimate" tools on rviz: the first is the tool for setting the initial pose of robot1, and the second is the tool for setting the initial pose of robot2. Use these two tools respectively to give the initial poses to the two robots.



- Then use the **2D Goal Pose** tool to give different goal points to the two robots: there are two "2D Goal Pose" tools on rviz, the first is the tool for setting the goal point of robot1, and the second is the tool for setting the goal point of robot2. Use these two tools respectively to give goal points to the two robots.



4. Source Code Analysis

4.1 multi_robot_base.launch

- File path

```
$HOME/microros_ws/src/microros_v2_bringup/launch/multi_robot_base.launch.py
```

`multi_robot_base.launch.py` is the chassis base launch file of the multi-machine system. It is responsible for creating independent namespaced nodes for each robot:

1. **Parameter parsing**: Parses the list of robot names from `robot_names` and the list of camera model types from `camera_types`; if only one camera type is passed in, it is automatically extended to all robots, and an error is reported when the counts do not match.
2. **Shared nodes**: `micro_ros_agent` (UDP 8090) launches only one instance globally (skipped if already running), and `multi_robot_sync` is responsible for distributing the keyboard/handle commands to each robot's `/cmd_vel`, `/beep`, and `/cmd_ptz_angle`.
3. **Independent nodes per robot**: Iterates over `robot_names` and creates the following for each robot:
    - `robot_state_publisher`: Loads the URDF via xacro, passes in `camera_type` and `namespace`, and publishes the TF transforms of that robot
    - `joint_state_publisher`: Subscribes to `ptz_status` and publishes the PTZ joint states
    - `ekf_filter_node`: Loads `ekf.yaml`, adds the namespace prefix to `odom_frame`, `base_link_frame`, `world_frame` as well as the input topics (such as `robot1/odom_raw`), enabling independent EKF fusion for each robot

```
def launch_setup(context, *args, **kwargs):
    # robot name list, e.g. "robot1,robot2"
    robot_names = [n.strip().strip('/') for n in

LaunchConfiguration('robot_names').perform(context).split(',') if n.strip()]

    # camera type per robot: ptz | no_ptz | basic_car
    camera_types = [c.strip() for c in

 LaunchConfiguration('camera_types').perform(context).split(',') if c.strip()]
    if len(camera_types) == 1:
        camera_types = camera_types * len(robot_names)
    if len(camera_types) != len(robot_names):
        raise RuntimeError(
            f'camera_types count ({len(camera_types)}) does not match
robot_names count ({len(robot_names)})')


 robot_urdf=os.path.join(get_package_share_directory('robot_description'),'urdf'
,'microros_v2.urdf.xacro')

 ekf_params=os.path.join(get_package_share_directory('microros_v2_bringup'),'con
fig','ekf.yaml')

    # single micro_ros_agent shared by all robots
    mircoros_agent=Node(
        package='micro_ros_agent',
        executable='micro_ros_agent',
```

```
        output='screen',
        arguments=['udp4', '--port', '8090', '-v',
LaunchConfiguration('micro_ros_agent_verbose')],
    )

    # single command-sync node, fans out cmd_vel/beep/cmd_ptz_angle to every
robot
    multi_robot_sync=Node(
        package='common_demo',
        executable='multi_robot_sync',
        name='multi_robot_sync',
        output='screen',
        parameters=[{'robot_names': robot_names}],
    )

    actions = [
        LogInfo(msg='micro_ros_agent already running, skip.') if
is_process_running('micro_ros_agent udp4') else mircoros_agent,
        multi_robot_sync,
    ]

    # per-robot nodes: robot_state_publisher / joint_state_publisher / ekf
    for name, camera_type in zip(robot_names, camera_types):
        prefix = f'{name}/'
        robot_description = Command(
            ["xacro", " ", robot_urdf," ", "camera_type:=", camera_type," ",
"namespace:=", name] )

        robot_state_publisher_node = Node(
            package='robot_state_publisher',
            executable='robot_state_publisher',
            namespace=name,
            parameters=[{'robot_description': ParameterValue(robot_description,
value_type=str)}],
            arguments=['--ros-args','--log-level','warn']
        )
        joint_publisher_node=Node(
            package='joint_state_publisher',
            executable='joint_state_publisher',
            namespace=name,
            parameters=[{'source_list': ["ptz_status"]}],
            arguments=['--ros-args','--log-level','warn']
        )

        #ekf_odom
        ekf_odom=Node(
            package='robot_localization',
            executable='ekf_node',
            name="ekf_filter_node",
            namespace=name,
            parameters=[ekf_params,
            {
                'odom_frame': f'{prefix}odom',
                'base_link_frame': f'{prefix}base_footprint',
                'world_frame': f'{prefix}odom',
                # chassis firmware topics are already namespaced
                'odom0': f'/{prefix}odom_raw',
                'imu0': f'/{prefix}imu',
```

```
            }],
            remappings=[('odometry/filtered', 'odom')]
        )

        actions += [
            robot_state_publisher_node,
            joint_publisher_node,
            ekf_odom,
        ]

    return actions


def generate_launch_description():
    # Declare arguments
    declared_arguments = []
    declared_arguments.append(
        DeclareLaunchArgument(
            'robot_names',
            default_value='robot1,robot2',
            description='Comma-separated robot namespaces, e.g. robot1,robot2'
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            'camera_types',
            default_value='ptz',
            description='Camera type per robot (ptz | no_ptz | basic_car):
single value for all robots, '
                        'or comma-separated list matching robot_names, e.g.
ptz,basic_car'
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            'micro_ros_agent_verbose',
            default_value='4',
            description='micro-ROS agent verbosity level (0-6)'
        )
    )

    return LaunchDescription([
        *declared_arguments,
        OpaqueFunction(function=launch_setup),
    ])
```

4.2 multi_robot_demo.launch

- File path

```
$HOME/microros_ws/src/microros_v2_bringup/launch/multi_robot_demo.launch.py
```

`multi_robot_demo.launch.py` is the launch file for multi-machine navigation. On the basis of `multi_robot_base.launch.py`, it loads the complete Nav2 navigation stack for each robot:

1. **Global map service**: Starts a `map_server` to load the shared grid map. All robots use the same map, which is automatically activated by `lifecycle_manager_map`.
2. **Dynamic rewriting of the parameter file**: The template file `nav2_multi_robot_params.yaml` contains the `<robot_ns>` placeholder. At startup, `ReplaceString` replaces it with the actual robot name, and then `root_key=name` wraps the parameters under the corresponding namespace, ensuring that `/robot1/controller_server` reads the configuration of `robot1`.
3. **Navigation nodes per robot**: Iterates over `robot_names` and starts the complete Nav2 navigation stack for each robot:
    - `amcl`: Localization node, subscribes to the global map topic `/map`, and outputs the pose under that robot's namespace
    - `controller_server`: Path tracking controller
    - `planner_server`: Global path planner
    - `behavior_server`: Behaviors such as obstacle avoidance and rotation
    - `bt_navigator`: Behavior tree navigation dispatcher

> The key difference from single-robot navigation: all robots share the same `map_server` and the global TF tree (no `/tf` remapping), and each robot's AMCL and costmaps are isolated by namespace without affecting each other.

```
    robot_names = [n.strip().strip('/') for n in

LaunchConfiguration('robot_names').perform(context).split(',') if n.strip()]
    map_file = os.path.join(os.path.expanduser('~'), 'map',
                            LaunchConfiguration('map').perform(context))
    bringup_dir = get_package_share_directory('microros_v2_bringup')
    params_template =
os.path.join(bringup_dir,'config','nav2_multi_robot_params.yaml')

    # single global map server shared by all robots
    actions = [
        Node(
            package='nav2_map_server',
            executable='map_server',
            name='map_server',
            output='screen',
            parameters=[{'yaml_filename': map_file, 'use_sim_time': False}],
        ),
        Node(
            package='nav2_lifecycle_manager',
            executable='lifecycle_manager',
            name='lifecycle_manager_map',
            output='screen',
            parameters=[{'autostart': True, 'node_names': ['map_server']}],
        ),
    ]

    lifecycle_nodes = ['amcl', 'controller_server', 'planner_server',
                       'behavior_server', 'bt_navigator', 'waypoint_follower']
    for name in robot_names:
```

```
        # rewrite '<robot_ns>' TF-frame placeholders, then wrap the file under
the
        # namespace root key so params match /robotN/xxx node names
        params_file = ParameterFile(
            RewrittenYaml(
                source_file=ReplaceString(
                    source_file=params_template,
                    replacements={'<robot_ns>': name}),
                root_key=name,
                param_rewrites={},
                convert_types=True),
            allow_substs=True)

        # NOTE: no /tf remapping here, all robots share the global TF tree
        actions += [
            Node(
                package='nav2_amcl',
                executable='amcl',
                name='amcl',
                namespace=name,
                output='screen',
                parameters=[params_file],
                # humble amcl has no map_topic param, remap to the shared global
map
                remappings=[('map', '/map')],
            ),
            Node(
                package='nav2_controller',
                executable='controller_server',
                namespace=name,
                output='screen',
                parameters=[params_file],
            ),
            Node(
                package='nav2_planner',
                executable='planner_server',
                name='planner_server',
                namespace=name,
                output='screen',
                parameters=[params_file],
            ),
            Node(
                package='nav2_behaviors',
                executable='behavior_server',
                name='behavior_server',
                namespace=name,
                output='screen',
                parameters=[params_file],
            ),
            Node(
                package='nav2_bt_navigator',
                executable='bt_navigator',
                name='bt_navigator',
                namespace=name,
                output='screen',
                parameters=[params_file],
            ),
            Node(
```

```
                package='nav2_waypoint_follower',
                executable='waypoint_follower',
                name='waypoint_follower',
                namespace=name,
                output='screen',
                parameters=[params_file],
            ),
            Node(
                package='nav2_lifecycle_manager',
                executable='lifecycle_manager',
                name='lifecycle_manager_navigation',
                namespace=name,
                output='screen',
                parameters=[{'autostart': True, 'node_names': lifecycle_nodes}],
            ),
        ]
    return actions
```